```
    for (i=0;i<n;i++) {                          w = δF − B · s.
        for (sum=0.0,j=0;j<n;j++) sum += qr->qt[j][i]*t[j];
        w[i]=fvec[i]-fvcold[i]-sum;
        if (abs(w[i]) >= EPS*(abs(fvec[i])+abs(fvcold[i]))) skip=false;
        Don't update with noisy components of w.
        else w[i]=0.0;
    }
    if (!skip) {
        qr->qtmult(w,t);                         t = Q^T · w.
        for (den=0.0,i=0;i<n;i++) den += SQR(s[i]);
        for (i=0;i<n;i++) s[i] /= den;           Store s/(s · s) in s.
        qr->update(t,s);                         Update R and Q^T.
        if (qr->sing) throw("singular update in broydn");
    }
}
qr->qtmult(fvec,p);
for (i=0;i<n;i++)                                Right-hand side for linear equations is −Q^T · F.
    p[i] = -p[i];
for (i=n-1;i>=0;i--) {                           Compute ∇f ≈ (Q · R)^T · F for the line search.
    for (sum=0.0,j=0;j<=i;j++) sum -= qr->r[j][i]*p[j];
    g[i]=sum;
}
for (i=0;i<n;i++) {                              Store x and F.
    xold[i]=x[i];
    fvcold[i]=fvec[i];
}
fold=f;                                          Store f.
qr->rsolve(p,p);                                 Solve linear equations.
lnsrch(xold,fold,g,p,x,f,stpmax,check,fmin);
```

`lnsrch` returns new $\mathbf{x}$ and f. It also calculates `fvec` at the new $\mathbf{x}$ when it calls `fmin`.

```
test=0.0;                                        Test for convergence on function values.
for (i=0;i<n;i++)
    if (abs(fvec[i]) > test) test=abs(fvec[i]);
if (test < TOLF) {
    check=false;
    delete qr;
    return;
}
if (check) {                                     True if line search failed to find a new x.
    if (restrt) {                                Failure; already tried reinitializing the Jacobian.
        delete qr;
        return;
    } else {
        test=0.0;                                Check for gradient of f zero, i.e., spurious con-
        den=MAX(f,0.5*n);                            vergence.
        for (i=0;i<n;i++) {
            temp=abs(g[i])*MAX(abs(x[i]),1.0)/den;
            if (temp > test) test=temp;
        }
        if (test < TOLMIN) {
            delete qr;
            return;
        }
        else restrt=true;                        Try reinitializing the Jacobian.
    }
} else {                                         Successful step; will use Broyden update for next
    restrt=false;                                    step.
    test=0.0;                                    Test for convergence on δx.
    for (i=0;i<n;i++) {
        temp=(abs(x[i]-xold[i]))/MAX(abs(x[i]),1.0);
        if (temp > test) test=temp;
    }
```

```
            if (test < TOLX) {
                delete qr;
                return;
            }
        }
    }
    throw("MAXITS exceeded in broydn");
}
```

9.7.4 More Advanced Implementations

One of the principal ways that the methods described so far can fail is if **J** (in Newton's method) or **B** in (Broyden's method) becomes singular or nearly singular, so that $\delta\mathbf{x}$ cannot be determined. If you are lucky, this situation will not occur very often in practice. Methods developed so far to deal with this problem involve monitoring the condition number of **J** and perturbing **J** if singularity or near singularity is detected. This is most easily implemented if the QR decomposition is used instead of LU in Newton's method (see [2] for details). Our personal experience is that, while such an algorithm can solve problems where **J** is exactly singular and the standard Newton method fails, it is occasionally less robust on other problems where LU decomposition succeeds. Clearly implementation details involving roundoff, underflow, etc., are important here and the last word is yet to be written.

Our global strategies both for minimization and zero finding have been based on line searches. Other global algorithms, such as the *hook step* and *dogleg step* methods, are based instead on the *model-trust region* approach, which is related to the Levenberg-Marquardt algorithm for nonlinear least squares (§15.5). While somewhat more complicated than line searches, these methods have a reputation for robustness even when starting far from the desired zero or minimum [2].

CITED REFERENCES AND FURTHER READING:

Deuflhard, P. 2004, *Newton Methods for Nonlinear Problems* (Berlin: Springer).[1]

Dennis, J.E., and Schnabel, R.B. 1983, *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*; reprinted 1996 (Philadelphia: S.I.A.M.).[2]

Broyden, C.G. 1965, "A Class of Methods for Solving Nonlinear Simultaneous Equations," *Mathematics of Computation*, vol. 19, pp. 577–593.[3]

CHAPTER 10

Minimization or Maximization of Functions

10.0 Introduction

In a nutshell: You are given a single function f that depends on one or more independent variables. You want to find the value of those variables where f takes on a maximum or a minimum value. You can then calculate what value of f is achieved at the maximum or minimum. The tasks of maximization and minimization are trivially related to each other, since one person's function f could just as well be another's $-f$. The computational desiderata are the usual ones: Do it quickly, cheaply, and in small memory. Often the computational effort is dominated by the cost of evaluating f (and also perhaps its partial derivatives with respect to all variables, if the chosen algorithm requires them). In such cases the desiderata are sometimes replaced by the simple surrogate: Evaluate f as few times as possible.

An extremum (maximum or minimum point) can be either *global* (truly the highest or lowest function value) or *local* (the highest or lowest in a finite neighborhood and not on the boundary of that neighborhood). (See Figure 10.0.1.) Finding a global extremum is, in general, a very difficult problem. Two standard heuristics are widely used: (i) Find local extrema starting from widely varying starting values of the independent variables (perhaps chosen quasi-randomly, as in §7.8), and then pick the most extreme of these (if they are not all the same); or (ii) perturb a local extremum by taking a finite amplitude step away from it, and then see if your routine returns you to a better point, or "always" to the same one. More recently, so-called *simulated annealing methods* (§10.12) have demonstrated important successes on a variety of global extremization problems.

Our chapter title could just as well be *optimization*, which is the usual name for this very large field of numerical research. The importance ascribed to the various tasks in this field depends strongly on the particular interests of whom you talk to. Economists, and some engineers, are particularly concerned with *constrained optimization*, where there are a priori limitations on the allowed values of independent variables. For example, the production of wheat in the United States must be a non-

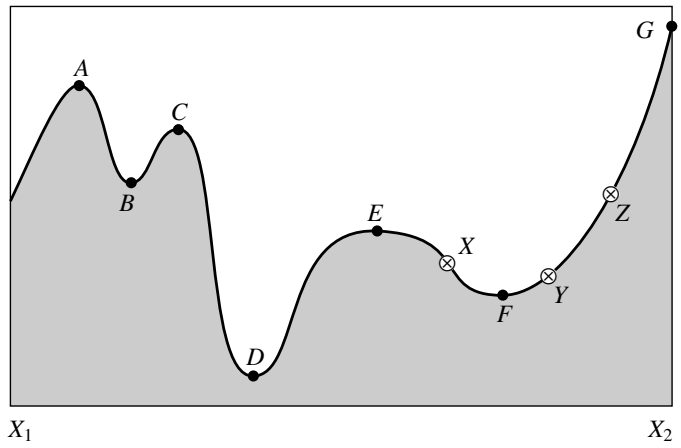

Figure 10.0.1. Extrema of a function in an interval. Points A, C, and E are local, but not global, maxima. Points B and F are local, but not global, minima. The global maximum occurs at G, which is on the boundary of the interval so that the derivative of the function need not vanish there. The global minimum is at D. At point E, derivatives higher than the first vanish, a situation that can cause difficulty for some algorithms. The points X, Y, and Z are said to *bracket* the minimum F, since Y is less than both X and Z.

negative number. One particularly well-developed area of constrained optimization is *linear programming*, where both the function to be optimized and the constraints happen to be linear functions of the independent variables. Sections 10.10 and 10.11, which are otherwise somewhat disconnected from the rest of the material that we have chosen to include in this chapter, discuss the two major approaches to such problems, the so-called *simplex algorithm* and *interior-point methods*.

Two other sections, §10.12 and §10.13, also lie outside of our main thrust, but for a different reasons. As mentioned, §10.12 discusses so-called *annealing methods*. These are stochastic, rather than deterministic, algorithms. Annealing methods have solved some problems previously thought to be practically insoluble: They address directly the problem of finding global extrema in the presence of large numbers of undesired local extrema. Section 10.13 discusses a different kind of minimization, namely that of path length along a directed graph by the technique known as *dynamic programming*. This will prove important later, in Chapter 16.

The other sections in this chapter constitute a selection of established algorithms for unconstrained minimization. (For definiteness, we will henceforth regard the optimization problem as that of minimization.) These sections are connected, with later ones depending on earlier ones. If you are just looking for the one "perfect" algorithm to solve your particular application, you may feel that we are telling you more than you want to know. Unfortunately, there is *no* perfect optimization algorithm. This is a case where we strongly urge you to try more than one method in comparative fashion. However, here are some guidelines:

For one-dimensional minimization (minimize a function of one variable), you must choose between methods that need only evaluations of the function, and methods that also require evalutations of the function's derivative. The latter are typically more powerful, but not always enough so as to compensate for the additional calculations of derivatives. We can easily construct examples favoring one approach or

the other.

- For one-dimensional minimization *without* calculation of the derivative, first bracket the minimum as described in §10.2, and then use *Brent's method* as described in §10.3. If your function has a discontinuous second (or lower) derivative, then the parabolic interpolations of Brent's method are of no advantage, and you might wish to use the simplest form of *golden section search*, as described in §10.2.
- For one-dimensional minimization *with* calculation of the derivative, §10.4 supplies a variant of Brent's method that makes limited use of the first derivative information. We shy away from the alternative of using derivative information to construct high-order interpolating polynomials. In our experience, the improvement in convergence very near a smooth, analytic minimum does not make up for the tendency of polynomials sometimes to give wildly wrong interpolations at early stages, especially for functions that may have sharp, "exponential" features.

For the multidimensional case, where you want to minimize a function of two or more variables, the analog of the derivative is the gradient, a vector quantity. You now have three options: compute the gradients using your function's known analytic form, compute the gradients by taking finite differences of computed function values, or don't compute the gradients at all. You also get to choose between methods that require storage of order N^2 and those that require only of order N, where N is the number of dimensions. For moderate values of N this is not a serious constraint; but if N is itself the number of points in a two- or three-dimensional grid, then N^2 storage may be prohibitive.

- For minimization without gradients, the *downhill simplex method* due to Nelder and Mead, discussed in §10.5, is slow, but sure. (This use of the word "simplex" is not to be confused with the simplex method of linear programming.) The method just crawls downhill in a straightforward fashion that makes almost no special assumptions about your function. While this can be extremely slow, it can also be extremely robust. Not to be overlooked is the fact that the code is concise and completely self-contained: a general N-dimensional minimization program in under 100 program lines! The storage requirement is of order N^2, and derivative calculations are not required.
- When your function has some smoothness to it, but you still don't want to compute gradients, turn to *direction set methods*, of which *Powell's method* is the prototype (§10.7). Powell's method requires a one-dimensional minimization subalgorithm such as Brent's method (see above). Storage is of order N^2. Direction set methods are much faster than the downhill simplex method. But keep reading for, possibly, an even better alternative.

Now the case where you are willing to calculate gradients from your function's known analytic form:

- *Conjugate gradient methods*, as typified by the *Fletcher-Reeves algorithm* and the closely related and probably superior *Polak-Ribiere algorithm* are widely used. Conjugate gradient methods require only of order a few times N storage, derivative calculations, and one-dimensional subminimization. Turn to §10.8 for detailed discussion and implementation.
- *Quasi-Newton* or *variable metric* methods are typified by the *Davidon-Fletcher-*

Powell (DFP) algorithm (sometimes referred to just as the *Fletcher-Powell* method) or the closely related *Broyden-Fletcher-Goldfarb-Shanno (BFGS)* algorithm. These methods require of order N^2 storage, derivative calculations, and one-dimensional subminimization. Details are in §10.9. Our personal experience is that the quasi-Newton methods dominate the conjugate gradient methods (if you can afford the storage), but there are probably applications where the reverse is true.

Finally, the case where the method uses gradients, but you are willing to let them be calculated by extra function evaluations (and finite differences):

- In our experience, the quasi-Newton (variable metric) methods work very well in this case, so much so that they can be significantly more efficient than Powell's method on suitable problems. In §10.9 we give an implementation that (almost) hides the gradient calculation completely. Thus quasi-Newton is effectively our first choice of method both when you are willing to calculate gradients, and when you are not willing!

You can now proceed to scale the peaks (and/or plumb the depths) of practical optimization.

CITED REFERENCES AND FURTHER READING:

Dennis, J.E., and Schnabel, R.B. 1983, *Numerical Methods for Unconstrained Optimization and Nonlinear Equations*; reprinted 1996 (Philadelphia: S.I.A.M.).

Polak, E. 1971, *Computational Methods in Optimization* (New York: Academic Press).

Gill, P.E., Murray, W., and Wright, M.H. 1981, *Practical Optimization* (New York: Academic Press).

Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), Chapter 17.

Jacobs, D.A.H. (ed.) 1977, *The State of the Art in Numerical Analysis* (London: Academic Press), Chapter III.1.

Brent, R.P. 1973, *Algorithms for Minimization without Derivatives* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2002 (New York: Dover).

Dahlquist, G., and Bjorck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), Chapter 10.

10.1 Initially Bracketing a Minimum

What does it mean to *bracket* a minimum? A root of a function is known to be bracketed by a pair of points, a and b, when the function has opposite sign at those two points. A minimum, by contrast, is known to be bracketed only when there is a *triplet* of points, $a < b < c$ (or $c < b < a$), such that $f(b)$ is less than both $f(a)$ and $f(c)$. In this case we know that the function (if it is smooth) has a minimum in the interval (a, c).

We consider the initial bracketing of a minimum to be an essential part of any one-dimensional minimization. There are some one-dimensional algorithms that do not require a rigorous initial bracketing. However, we would *never* trade the secure feeling of *knowing* that a minimum is "in there somewhere" for the dubious reduction of function evaluations that these nonbracketing routines may promise. Please bracket your minima (or, for that matter, your zeros) before isolating them!